COMPSEC300
Programming Project Report

[REDACTED]

Project Github: https://github.com/k-um-i/himitsu

秘密 (himitsu)

Noun, Na-adjective (keiyodoshi), Noun which may take the genitive case particle 'no'

1. secret; secrecy; confidentiality; privacy

Table of contents

General description

General information about the program

For the programming project I have implemented a relatively basic password manager program called *Himitsu*. The main purpose of Himitsu is to provide a secure and simple to use local password manager that is also lightweight and portable. I believe these goals have been realized by using web-ui for the interface implementation which allows the interface implementation to be lightweight and highly compatible due to the utilization of already existing web browsers, and the use of encrypted XML files for the password databases, allowing for easy transferring of database files.

The background logic used by the program is written in *GOLang*, while the frontend through which the user interacts with the program, is implemented using the *go web-ui* library which utilizes *HTML* and *Javascript* to interact with the backend of the program. For a more thorough look at the program structure and implementation, refer to the *Structure of the program* section of the report.

Basic usage

The basic usage of Himitsu should be quite simple and user friendly once you have the program up and running. For instructions on the installation of the program, either through precompiled binaries or building the program yourself from source, please refer to the *Installation* section I have written on the project github. Do note that the Himitsu executable must be located in the same directory with the *index.html* file and the directories *javascript* and *themes*. There is also an usage guide on the project github, but it does not go over how to use the GUI.

It should also be noted that a web browser must be installed on the system in order to launch the web-ui interface of Himitsu.

Launching the GUI

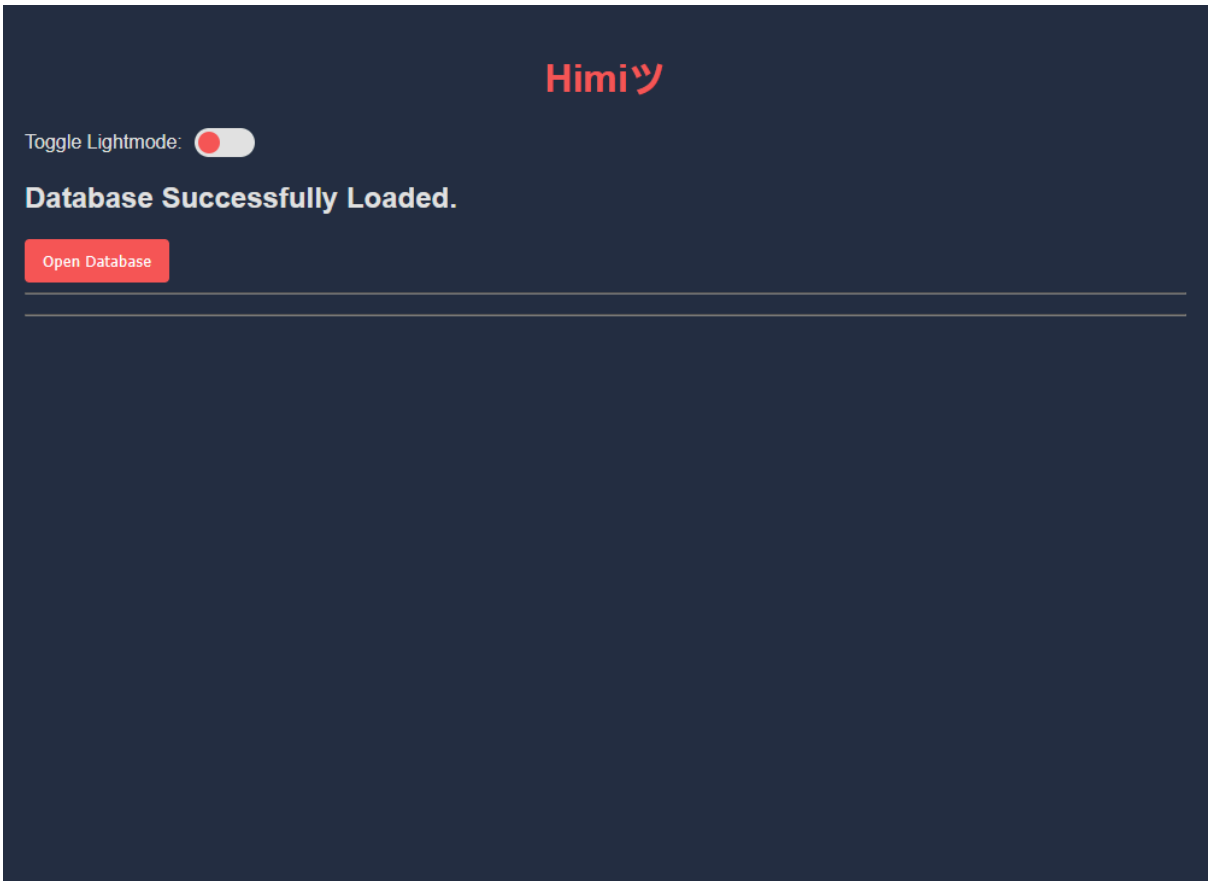
To launch Himitsu, start by opening a terminal window in the project directory. There are currently three terminal commands that Himitsu supports. They are as follows:

- Create and open a new empty password database:
  `./himitsu create <filename>`
- Open an existing password database:
  `./himitsu open <filename>`
- Change the password of an existing password database:
  `./himitsu passwd <filename>`

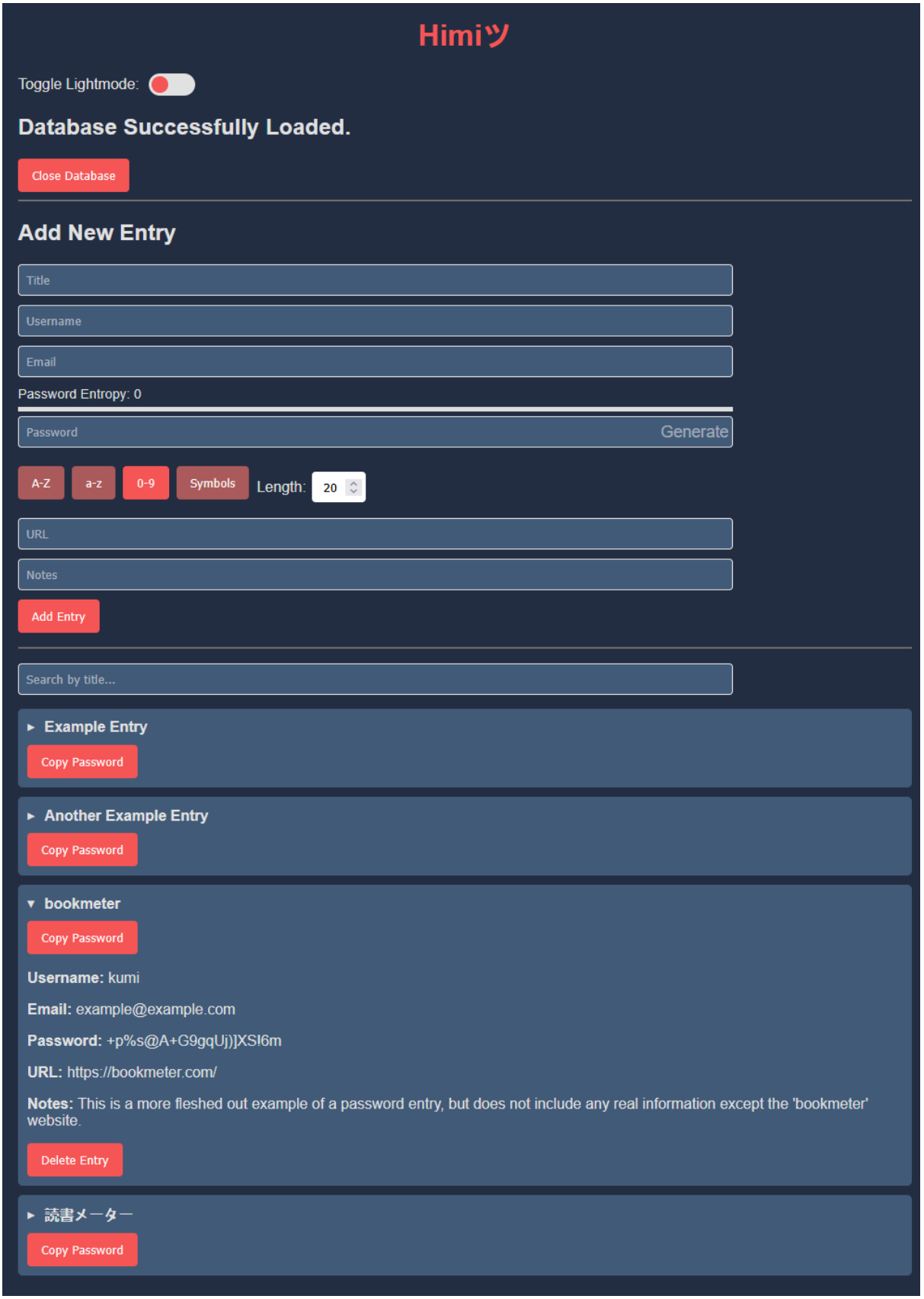
The third command will not launch the GUI, but will instead only perform the password change for the database. You can of course open the GUI afterwards by running the command *open* on the database you have just modified using *passwd*.

Using the GUI

Whenever you launch the Himitsu GUI you will be greeted with the following screen:



Simply click the *Open Database* button to reveal the rest of the GUI. In this view you can also see the *Toggle Lightmode* -toggle. This toggle can be used to change the theme of Himitsu to a lightmode one. The CSS for both the darkmode and lightmode themes can be found in the *themes* directory.



After you open the database you will see the rest of the user interface. This will be the general view you will be using Himitsu through. In case you have just created your own empty password database the bottom part will be empty until you add your own entries.

You can minimize the database view by clicking on the *Close Database* button.

To add a new entry to your database you can fill out the fields under the *Add New Entry* section. The *Add New Entry* section also includes the random password generator implemented in Himitsu. You can choose the characters used and the length of the password simply by toggling the options under the *Password* field. Once you click *Generate* inside the *Password* field your new password will be generated according to your settings. The strength of the password will also be displayed above the *Password* field at all times. This is based on the entropy of the entered password.

Note that only the *Title* field is required and you can, for example, save only the password and title inside an entry if you so wish. Once you are done adding information to your entry, simply click the *Add Entry* button and your new entry will be added to the database. Entry rendering will be handled in a way where only the fields with information stored will be visible inside the entry details.

You can browse through the entries you have saved inside your database at the bottom of the GUI view. There is also a *Search by title…* field that you can use to filter your saved entries based on their title. The *Copy Password* button will copy the password of that specific entry to your clipboard. To view the full details of an entry simply click it to bring out a drop down menu that will display all of the information, including a *Delete Entry* button. By clicking the *Delete Entry* button you can delete an entry from your database.

The database is updated in real time as you interact with the GUI which means that you don't need to worry about saving the changes you have made to the database. Once you are done, simply close the GUI window and the backend will automatically terminate its process as well.

Structure of the program

Backend

The backend of Himitsu consists mainly of GOLang code and in part Javascript that is used as a bridge between the GOLang backend and HTML frontend. The GO implementation consists of two packages; *encryption* & *gui,* and the *main.go* file, while all of the Javascript code is written in a single *script.js* file.

Main.go

The *main.go* file implements the functionality that the user directly interacts with through the command line. It has a single function *main()* which handles the user input through arguments when running the executable. It implements the three commands that I have already gone over earlier in the *Launching the GUI* section of the report.

Whenever the *main.go* file requests the user for a password, the input is handled securely by the *ReadPassword()* function from the *golang.org/x/term* GOLang library that ensures the user's password is handled securely and read without local echo.

Encryption

The encryption used by Himitsu is implemented in the *encryption/encryption.go* file. This file also includes some other cryptography related function implementations such as *DeriveKey(), UpdateDatabase()* and *GenerateSecurePassword()*. The encryption scheme used by Himitsu is 256-bit AES GCM encryption and the encryption key is derived from the user's master password using the argon2 hashing algorithm.

The following parameters are used for encryption:

- 12 Byte random IV for AES
- 16 Byte random salt for key derivation
- 32 Byte encryption key derived from salt and random password

All random numbers are generated cryptographically secure by using the GOLang library *crypto/rand*. The same secure random number generator is used for the random password generation as well.

GUI

The GOLang backend that is directly connected to the frontend GUI is implemented in the *gui/gui.go* file. This file implements multiple helper functions that are then bound to the web-ui GUI and called from the frontend. These functions can then call the necessary functions from *encryption/encryption.go* to perform the necessary database operations.

An important function implemented entirely inside the GUI file is the *passStrn()* function. This function is used to calculate the entropy of the password currently inputted inside the *Password* field under the *Add New Entry* section of the user interface. For calculating the password entropy I am using the *go-password-validator* library, which is a lightweight library used for determining the strength of a password. It has some weaknesses since it is designed to run without requiring large amounts of information about common passwords, but I deemed it to be enough for this project.

The launching of the GUI is also implemented in this file under the *StartGui()* function. This function creates the new window, binds all of the helper functions to it and starts up the GUI window. The function also waits for the window to be closed to tell the rest of the program to shutdown.

Javascript

The javascript inside Himitsu is used as a bridge between the GOLang backend and HTML frontend and implemented inside the *javascript/script.js* file. It implements functions that are called by the HTML code and in turn call the bound helper functions from *gui/gui.go*.

In addition to implementing functions that interact with the GOLang backend, there are also some javascript functions implemented that perform necessary actions on their own, such as the *renderEntries()* function which dynamically adds the entries inside a database to the GUI. Other functions that also interact more directly with the GUI such as closing the database or switching UI themes are mostly implemented fully in javascript inside the *javascript/script.js* file.

Frontend

The frontend is fully implemented using HTML inside the *index.html* file, located in the base directory of Himitsu. This HTML file is opened inside a web-view window when the web-ui GUI implementation gets called from the *gui/gui.go* file. The frontend consists of basic HTML elements that make-up the user interface that the user can interact with. Part of the HTML display is handled by the Javascript implementation, mainly the dynamic rendering of password entries inside the opened database and the showing/hiding of specific UI elements.

Secure programming solutions and testing

Jenkins

Jenkins is the main program I have used for trying to keep the code secure. I have had a Jenkins pipeline setup since starting the project. The pipeline is used for both dependency vulnerability checking and static code analysis.

The dependency vulnerability checking is conducted using *Nancy*, which utilizes the *Sonatype OSS Index* for checking any dependencies used by Himitsu for any vulnerabilities.

Static code analysis is performed using *gosec* which scans the himitsu code that is written in GOLang and looks for any possible security problems with the code.

I have not had any issues with vulnerable dependencies when it comes to Himitsu, so I have not had to modify anything because of the results of Nancy. Nancy has only ever returned one dependency as vulnerable, but it had no vulnerabilities that would cause issues with Himitsu, since they were all related to vulnerabilities against websites.

However, gosec has returned multiple types of possible security issues during the development of Himitsu. Most of them have been security problems that aren't actually present in the code such as: "Use of hardcoded IV/nonce for encryption by passing hardcoded slice/array", because even though the IV is saved inside a variable, it is still securely randomly generated each time encryption is performed.

A limitation with the current implementation of the pipeline used for testing, is the absence of any sort of automatic testing on the HTML/Javascript code of Himitsu. This is an oversight by me, because when I wrote the pipeline code I hadn't yet decided on what kind of GUI I would be implementing for Himitsu so I didn't know I would be utilizing such a large amount of HTML and Javascript code.

Here is the pipeline code used during development:

```
pipeline {
	agent any
	environment {
	GOROOT = "/usr/lib/go"
	GOPATH = "${WORKSPACE}/go"
	PATH = "${GOROOT}/bin:${GOPATH}/bin:${env.PATH}"
	}
	stages {
	stage('Checkout') {
		steps {
		git branch: 'trunk',
			url: 'https://github.com/k-um-i/COMPSEC300_PM.git'
		}
	}
	stage('Setup Go Environment') {
		steps {
		sh 'go version'
		sh 'go mod tidy'
		}
	}
	stage('Static Code Analysis (gosec)') {
		steps {
		sh 'gosec -no-fail -exclude-dir=pkg -fmt=json -out gosec.json
./...'
		}
	}
	stage('Dependency scan (nancy)') {
		steps {
		script {
			try {
				sh 'touch nancy.json'
				sh 'go list -m all | nancy sleuth --skip-update-check
-o json-pretty > nancy.json'
			} catch (err) {
				echo err.getMessage()
			}
		}
		}
	}
	stage('Archive Results') {
		steps {
		archiveArtifacts artifacts: 'gosec.json, nancy.json', fingerprint:
true
		}
	}
	}
}
```

Manual testing

During the development of Himitsu I have personally been constantly testing the program and trying to find any possible issues with the code. However I know that this is not very sufficient which is why I had implemented the Jenkins pipeline. Towards the end of development I also thought it a good idea to have a few friends test out the program before I added the final touches to the code.

I sent the first alpha build of the program to two of my friends. Both could get it running correctly and were able to utilize the basic functions of the password manager. One of my friends found a massive oversight I had made which allowed direct HTML injection into password entries when adding new entries through the GUI. I also got a suggestion to require a title for the password entry and to only include information fields with stored information in the details of each rendered entry.

Both of these suggestions were implemented in the final version of Himitsu and I also managed to add proper input sanitization to prevent HTML injections.

Incomplete implementations

Jenkins pipeline

The Jenkins pipeline should be updated to take into account both HTML and Javascript code analysis. It is currently outdated since it was written before go-webui was part of the Himitsu password manager.

Suggestions for improvements

While I do believe that Himitsu is currently in a state where it fulfils the basic requirements of acting as a password manager, I still think it is missing some important features that could be implemented in the future, both security related and just general feature related.

When it comes to security related features, something that I was originally planning to implement to the program, but ended up not having time for was having keyfile support for the derivation of the encryption key in addition to a master password. This would have allowed the user to add a file as part of their encryption key for a password database, similar to the implementation included in *VeraCrypt*.

In terms of general feature improvements there are plenty that could be made such as adding categories and category based sorting for the password entries, cleaning up the UI and making it more visually pleasing in general, adding the ability to edit existing password entries, adding expiration dates for password entries that would remind the user to change their password periodically, etc. etc.